

EC-341 Digital System Design

BS (CE)-2021

LAB REPORT #1



Submitted By:

Reg. No.	Name
21-CE-009	Huzaifa Shahbaz

Department of Computer Engineering

HITEC University Taxila

	Presentation (Format(0.5)+ Title (0.5)+ Conclusion(1) +Procedure(2) +Objective(1))	Calculation/ Coding	Observation/ Program Results	Total
Total	5	5	5	15
Obtained				

Lab Instructor.

Fasih Ahmad

Experiment #1

Introduction to Verilog and ModelSim

Objective:

The objectives of an Introduction to Verilog and ModelSim course include equipping students with the knowledge and skills needed to design, simulate, verify, and debug digital circuits using Verilog HDL and ModelSim simulation tools.

Equipment/Software Used:

ModelSim

Procedure:

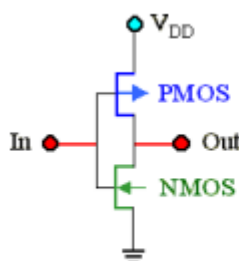
Introduction to Verilog:

One HDL (Hardware Description Language) is Verilog. Digital electronic circuits are modeled and simulated with its help. A design can be synthesized to create gate level designs, which are subsequently translated to physical design, once it has been tested, simulated, and prepared for "tape-out" to the fabrication facility. Verilog HDL is sometimes thought of as the code that embodies the greatest level abstraction available for electronic circuits. It differs from general-purpose programming languages in that hardware modeling is its exclusive aim. Thus, you can specify registers, wires, gates, clocks, etc. in Verilog. When you want to simulate and verify hardware specifications before synthesizing the circuit, it is quite helpful and can save a lot of time.

Verilog Abstraction Level

1. Switch (Transistor) Level:

The switch level, which refers to the capacity to represent the circuit as a netlist of transistor switches, is the lowest level of abstraction for a digital HDL. A more comprehensive modeling technique that can identify a few more electrical issues when transistors are utilized in this manner. Little-used now that most circuits aren't constructed in this manner.



```
module cmos_inv (In, Out); // module declaration
input In;
output Out;
supply1 vdd;
supply0 gnd;
    pmos p1(Out, vdd, In); // pmos instantiation
    nmos n1(Out, gnd, In); // nmoc instantiation
endmodule
```

2. Gate (Structural) Level:

The capacity to represent the circuit as a netlist of simple logic gates and functions is known as the gate level, which is a marginally higher abstraction. The gates have multiple scalar

inputs and one scalar output.

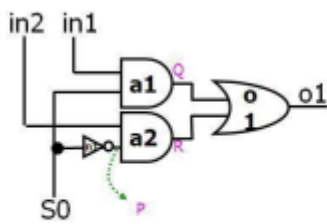


Fig. Multiplexer 2:1

```

module mux2_1 (o1,in1,in2,S0); //module
output o1;
input in1,in2,S0;
and a1 (Q, in1, S0); //and instantiation
not n1 (P,S0);      // not instantiation
and a2 (R, in2, P); // and instantiation
or o1 (o1, Q, R); // or instantiation
endmodule

```

Verilog Code

3.Data Flow Level:

The gate level modeling approach is particularly effective for small circuits because it allows the designer to instantiate and link each gate separately. On the other hand, a complicated design has a very high gate count. It is therefore wise to implement the function at a level above the gate level. With the advancement of logic synthesis tools, dataflow modeling has gained popularity as a design methodology. With this method, the circuit designer can focus on improving data flow within the circuit.

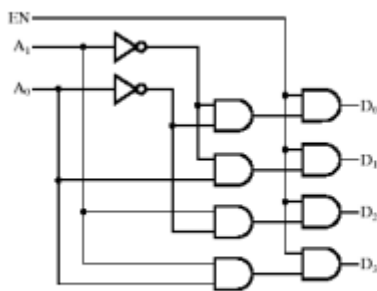


Fig. Line Decoder (2-4)

```

module deco2_4 (EN, A0, A1, D0, D1, D2,
D3); //module
input EN, A0, A1;
output D0, D1, D2, D3;
  assign D0 =(EN & ~A1 & ~A0);
  assign D1 =(EN & ~A1 & A0);
  assign D2 =(EN & A1 & ~A0);
  assign D3 =(EN & A1 & A0);
endmodule

```

Verilog Code

4. Functional and Algorithmic Level (Behavioral Modeling):

The designer can express the functionality of the design in an algorithmic way with Verilog. Stated differently, the circuit designer explains how the circuit behaves. Writing the logic in the C language is the abstraction used in this modeling. Procedural statements in Verilog behavioral models are used to alter variables of the previously mentioned data kinds and control the simulation. The activity always begins at the initial control construct. A distinct activity flow is initiated by each always statement and each starting statement. Every activity flow is concurrent, enabling the user to simulate the hardware's intrinsic concurrency.

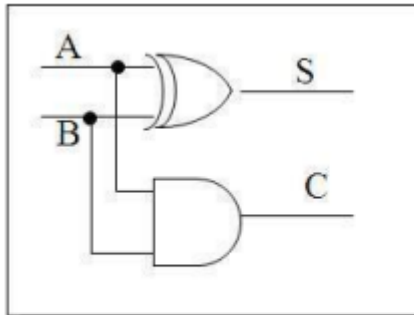


Fig. Binary counter(8-bit)

```
module half_add (S, C, A, B);
  input A, B;
  output S, C;
  reg S, C;
  always @ (A or B)
  begin
    S = A ^ B;
    C = A & B;
  end
end
```

Verilog Code

5. Register transfer Level (RTL Modelling):

Describes the data and control signals that flow within and across functional units to describe a system. Using HDL models of registered circuits and the signals that flow between them—such as memory, flip flops, latches, shift registers, and so forth—you can create behavioral design concepts. Behavioral and data flow modeling are combined in RTL, which ought to be synthesizable. Compared to behavior hardware description, RTL description is less dependent on technology and more sophisticated.

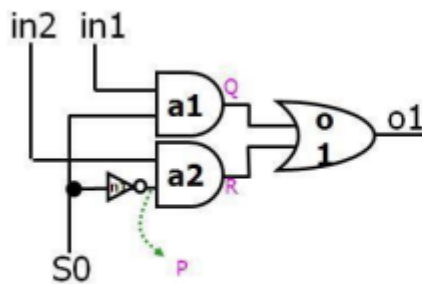


Fig. Multiplexer 2:1

```
module mux2to1 (in1,in2, S0, o1);
  input in1,in2,S0;
  output o1;
  always_comb begin
    case (sel)
      1'b0 : o1 = in1;
      1'b1 : o1 = in2;
      default : o1 = 1'bx;
    endcase
  end
end
```

Verilog Code

Tasks

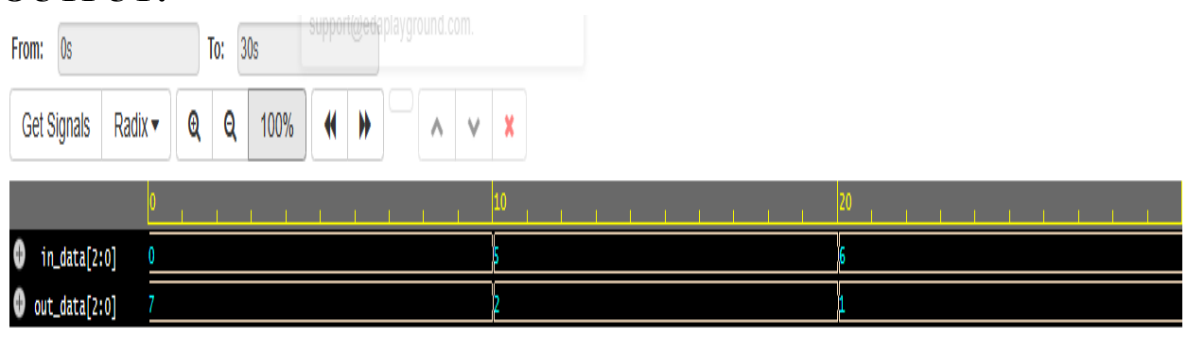
Task #1:

Write a Gate Level Verilog code to find 1's complement of three-bit numbers.

CODE:

```
1 module ones_complement (input [2:0] in_data,output reg [2:0] out_data);
2   always @(*) begin
3     out_data = ~in_data;
4   end
5 endmodule
6 module testbench;
7   reg [2:0] in_data;
8   wire [2:0] out_data;
9   ones_complement dut (
10    .in_data(in_data),
11    .out_data(out_data)
12  );
13  initial
14  begin
15    $monitor("Input: %b, Output: %b", in_data, out_data);
16    // Test cases
17    in_data = 3'b000;
18    #10;
19    in_data = 3'b101;
20    #10;
21    in_data = 3'b110;
22    #10;
23    $finish;
24  end
25 endmodule
```

OUTPUT:



Task #2:

Write a Gate Level Verilog code for Half Adder and write code for test bench as well. Simulate the Adder for different set of inputs. Attach waveform screenshots and Verilog code.

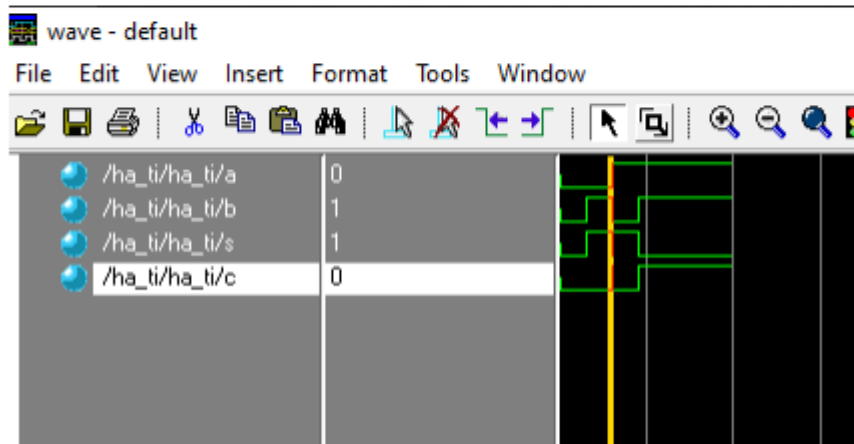
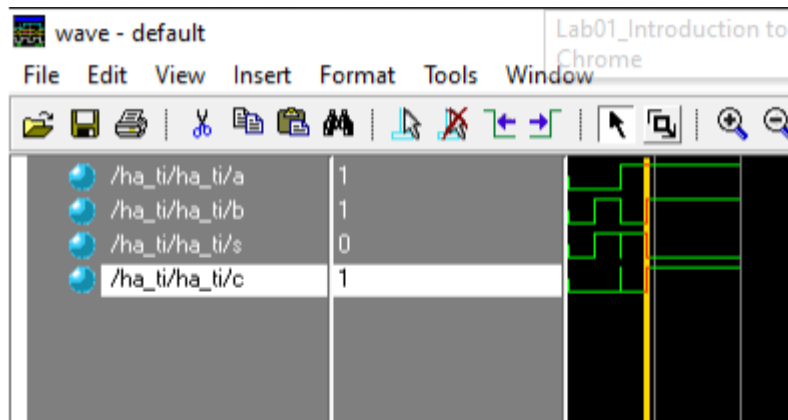
CODE:

```

1 module half_add(a,b,sum,carry);
2 input a,b;
3 output sum,carry;
4
5 xor x1(sum,a,b);
6 and al(carry,a,b);
7 endmodule
8
9 module h_a_ti();
10 reg x,y;
11 wire ss,cc;
12
13 half_add h_a_ti(x,y,ss,cc);
14 initial
15 begin
16 x=0;y=0;
17 #30; x=0;y=1;
18 #30; x=1;y=0;
19 #30; x=1;y=1;
20 end
21 endmodule

```

OUTPUT:



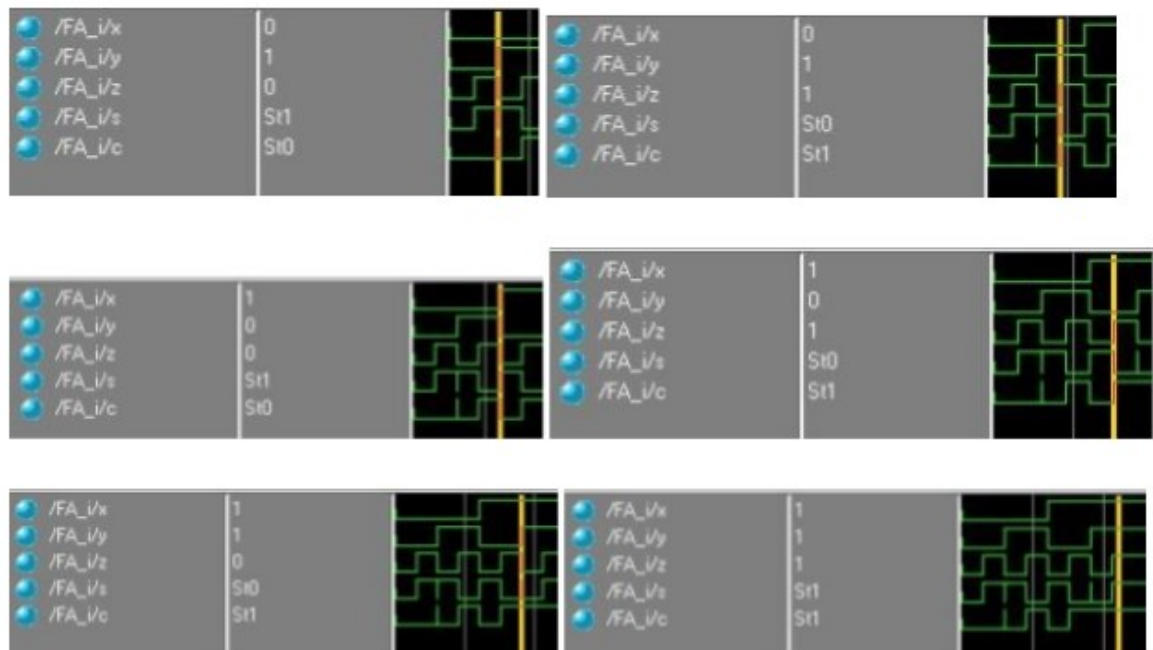
Task #3:

Write code for above task for Full Adder as well.

CODE:

```
1 module FA (
2     input a,b,cin,
3     output s,
4     output c
5 );
6 wire x1,x2,x3;
7 xor (x1,a,b);
8 and (x3,a,b);
9 xor (s,x1,cin);
10 and (x2,x1,cin);
11 or (c,x2,x3);
12 endmodule
13
14 module FA_i ();
15 reg x,y,z;
16 wire s,c;
17 FA al (x,y,z,s,c);
18 initial
19 begin
20 x=0;y=0;z=0;
21 #30 x=0;y=0;z=1;
22 #30 x=0;y=1;z=0;
23 #30 x=0;y=1;z=1;
24 #30 x=1;y=0;z=0;
25 #30 x=1;y=0;z=1;
26 #30 x=1;y=1;z=0;
27 #30 x=1;y=1;z=1;
28 end
29 endmodule
```

OUTPUT:



Conclusion:

In conclusion, the introduction to ModelSim and Verilog offers a comprehensive initiation into digital design, showcasing gate-level implementations like half adder, full adder, and one's complement arithmetic, laying a robust foundation for hardware description and simulation proficiency.